Práctica: Desarrollo con agentes

Parte 2 y Parte 3

Calculadora de notas con tools y GitHub MCP

Fecha: 29/04/2026

Apartado	Contenido
Planificación	PRD, SPEC y definición de agentes.
Tools	Separación del cálculo en `tools_notas.py`.
Desarrollo	Adaptación de `main.py` y pruebas unitarias.
Verificación	Ejecuciones reales de media y mediana.
Publisher	Publicación real en GitHub y uso comprobado de GitHub MCP.

En esta práctica se amplía la calculadora de notas para separar la lógica de cálculo en funciones externas y documentar la publicación del proyecto junto con el uso real de GitHub MCP.

1. Introducción

La aplicación sigue siendo sencilla: pide el nombre del alumno, tres notas y el tipo de cálculo. La diferencia principal está en que la media y la mediana ya no se resuelven dentro del programa principal, sino mediante una tool externa en Python.

- Se separa la lógica de cálculo en `tools_notas.py`.
- El usuario puede elegir entre media o mediana.
- La entrega documenta la publicación real en GitHub y una acción ejecutada mediante GitHub MCP.

2. PRD

Objetivo

Crear una aplicación de consola en Python que permita introducir el nombre de un alumno y tres notas, elegir si se quiere calcular la media o la mediana, y mostrar si el resultado es aprobado o suspenso.

Usuario

Profesorado o alumnado que quiera calcular rápidamente una nota final usando un flujo de trabajo organizado con agentes.

Funcionalidades

- Pedir el nombre del alumno
- Pedir tres notas numéricas
- Permitir elegir entre media o mediana
- Calcular el resultado numérico usando funciones externas en `tools_notas.py`
- Mostrar el resultado numérico final
- Indicar aprobado si el resultado es mayor o igual que 5
- Indicar suspenso si el resultado es menor que 5
- Publicar el proyecto en GitHub
- Dejar una evidencia real del uso de GitHub MCP sobre el repositorio

Requisitos técnicos

- Implementación en Python
- Ejecución por consola
- Separación de la lógica de cálculo en un archivo de tools
- Código claro y sencillo
- Documentación del uso de agentes y de GitHub MCP

3. SPEC

Problema

Se necesita una aplicación sencilla para calcular la nota final de un alumno a partir de tres notas. El usuario podrá elegir si quiere calcular la media o la mediana. Además, el proyecto debe quedar publicado en GitHub y acompañado por una prueba real del uso de GitHub MCP sobre el repositorio.

Solución esperada

El programa solicitará:

- nombre del alumno
- nota 1
- nota 2
- nota 3
- tipo de cálculo: `media` o `mediana`

Después:

- usará `tools_notas.py` para calcular el resultado numérico
- mostrará el nombre del alumno
- mostrará el tipo de cálculo elegido
- mostrará el resultado numérico
- mostrará `Aprobado` o `Suspenso`

Archivos principales esperados

- `main.py`
- `tools_notas.py`
- `test_main.py`
- `PRD.md`
- `SPEC.md`
- `AGENTS.md`
- `README.md`

Criterios de aceptación

- `calcular_media(notas)` devuelve la media correcta
- `calcular_mediana(notas)` devuelve la nota central tras ordenar tres notas
- `main.py` importa y usa `calcular_media` y `calcular_mediana`
- El programa permite elegir entre media o mediana
- Si el resultado numérico es mayor o igual que 5, muestra `Aprobado`

- Si el resultado numérico es menor que 5, muestra `Suspenso`
- El proyecto queda documentado y preparado para GitHub
- La entrega incluye evidencia real del uso de GitHub MCP

4. Organización del trabajo

Reglas generales

- Todos los agentes deben basarse en `PRD.md` y `SPEC.md`.
- No inventar funcionalidades no pedidas.
- Mantener el código simple y claro.
- Explicar brevemente los cambios realizados.

Agent 1: Architect

Rol:

- Leer `PRD.md` y `SPEC.md`.
- Resumir la solución.
- Proponer la estructura mínima del programa y la separación en archivos.

Salida esperada:

- Explicación breve de la estructura del proyecto.

Agent 2: Builder

Rol:

- Implementar y modificar `main.py`.
- Seguir exactamente la SPEC.
- Integrar el uso de `tools_notas.py`.

Salida esperada:

- Archivo `main.py` funcional.

Agent 3: Tester

Rol:

- Revisar si `main.py` y `tools_notas.py` cumplen los criterios de aceptación.
- Proponer casos de prueba manuales.
- Detectar errores o mejoras.

Salida esperada:

- Lista de pruebas y verificación.

Agent 4: Documenter

Rol:

- Redactar `README.md`.

- Explicar qué hace el programa y cómo ejecutarlo.

Salida esperada:

- Documentación breve y clara del proyecto.

Agent 5: Tool Specialist

Rol:

- Crear funciones externas para calcular la media y la mediana.
- Separar la lógica de cálculo del programa principal.
- Verificar que las funciones reciben una lista de notas.
- No añadir funcionalidades innecesarias.

Salida esperada:

- Archivo `tools_notas.py`.
- Explicación breve de las funciones creadas.

Agent 6: Publisher

Rol:

- Preparar el proyecto para publicarlo en GitHub.
- Revisar que existen los archivos principales.
- Usar GitHub MCP sobre el repositorio para dejar una evidencia real.
- Crear un resumen del proyecto.
- Crear y mantener un issue con mejoras futuras.

Salida esperada:

- Comprobación final de archivos.
- Proyecto publicado en GitHub.
- Evidencia del uso de GitHub MCP.
- Issue con mejoras.

5. Tool Specialist y desarrollo

Primero se definió la lógica del cálculo en una tool externa y después se integró en `main.py`.

5.1. Evidencia del Tool Specialist

Prompt

Actúa como Tool Specialist.

Basándote en `PRD.md` y `SPEC.md`, crea un archivo `tools_notas.py` con dos funciones:

1. `calcular_media(notas)`

2. `calcular_mediana(notas)`

Requisitos:

- `notas` será una lista de tres números
- `calcular_media` devolverá la media aritmética
- `calcular_mediana` devolverá la nota central después de ordenar
- código sencillo
- sin funcionalidades extra

Respuesta

Se creó `tools_notas.py` con:

- una función `validar_notas(notas)` para comprobar que llega una lista de tres valores numéricos
- `calcular_media(notas)` para devolver la media
- `calcular_mediana(notas)` para devolver la nota central

Resultado

La lógica de cálculo quedó separada del programa principal, tal como pedía el enunciado.

5.2. Archivo tools_notas.py

```
"""Funciones externas para calcular la media o la mediana de tres notas."""


def validar_notas(notas: list[float]) -> None:
    """Comprueba que las notas llegan en una lista de tres valores numéricos."""
    if not isinstance(notas, list):
        raise ValueError("Las notas deben recibirse en una lista.")

    if len(notas) != 3:
        raise ValueError("Se requieren exactamente tres notas.")

    for nota in notas:
        if not isinstance(nota, (int, float)):
            raise ValueError("Cada nota debe ser numérica.")


def calcular_media(notas: list[float]) -> float:
    """Devuelve la media aritmética de tres notas."""
```

```
    validar_notas(notas)
    return sum(notas) / len(notas)


def calcular_mediana(notas: list[float]) -> float:
    """Devuelve la nota central tras ordenar tres notas."""
    validar_notas(notas)
    notas_ordenadas = sorted(notas)
    return notas_ordenadas[1]
```

5.3. Evidencia del Builder

Prompt

Actúa como Builder.

Modifica `main.py` para usar las funciones del archivo `tools_notas.py`.

Requisitos:

- importar `calcular_media` y `calcular_mediana`
- pedir nombre del alumno
- pedir tres notas
- preguntar si se quiere calcular media o mediana
- mostrar el resultado numérico
- indicar aprobado si el resultado es mayor o igual que 5
- indicar suspenso si es menor que 5
- mantener el código sencillo

Respuesta

Se adaptó `main.py` para:

- importar las funciones externas
- pedir tres notas por consola
- preguntar el tipo de cálculo
- delegar el cálculo en `tools_notas.py`
- mostrar el resultado final

Resultado

`main.py` quedó conectado con `tools_notas.py` y cumple la SPEC.

6. Código principal y pruebas

6.1. Archivo main.py

```python
"""Calculadora de notas con uso de tools externas."""

from tools_notas import calcular_media, calcular_mediana


def parse_grade(value: str) -> float:
    """Convierte una entrada en nota numérica y valida su rango."""
    try:
        grade = float(value)
    except ValueError as exc:
        raise ValueError("La nota debe ser un número.") from exc

    if grade < 0 or grade > 10:
        raise ValueError("La nota debe estar entre 0 y 10.")

    return grade


def determine_result(score: float) -> str:
    """Devuelve Aprobado o Suspenso según el resultado numérico."""
    return "Aprobado" if score >= 5 else "Suspenso"


def request_calculation_option() -> str:
    """Pide si se quiere calcular la media o la mediana."""
    while True:
        option = input("¿Qué quieres calcular? media/mediana: ").strip().lower()
        if option in {"media", "mediana"}:
            return option
        print("Opción no válida. Escribe 'media' o 'mediana'.")


def request_grades() -> list[float]:
    """Solicita tres notas válidas por consola."""
    grades = []
    for index in range(1, 4):
        while True:
            try:
                grade = parse_grade(input(f"Introduce la nota {index}: ").strip())
                grades.append(grade)
                break
            except ValueError as error:
                print(f"Entrada inválida: {error}")
    return grades


def main() -> None:
    """Ejecuta el flujo principal del programa."""
    student_name = input("Introduce el nombre del alumno: ").strip()
    while not student_name:
        student_name = input("El nombre no puede estar vacío. Introduce el nombre del alumno: ").strip()

    grades = request_grades()
    option = request_calculation_option()

    if option == "mediana":
        numeric_result = calcular_mediana(grades)
    else:
        numeric_result = calcular_media(grades)

    result = determine_result(numeric_result)

    print("\nAlumno:", student_name)
```

```
    print("Tipo de cálculo:", option)
    print("Resultado numérico:", round(numeric_result, 2))
    print("Resultado:", result)


if __name__ == "__main__":
    main()
```

6.2. Archivo test_main.py

```
import unittest

from main import determine_result, parse_grade
from tools_notas import calcular_media, calcular_mediana, validar_notas


class TestToolsNotas(unittest.TestCase):
    def test_calcular_media(self):
        self.assertAlmostEqual(calcular_media([5, 5, 5]), 5.0)
        self.assertAlmostEqual(calcular_media([10, 8, 9]), 9.0)

    def test_calcular_mediana(self):
        self.assertEqual(calcular_mediana([2, 10, 4]), 4)
        self.assertEqual(calcular_mediana([9, 3, 7]), 7)

    def test_validar_notas_requires_list(self):
        with self.assertRaises(ValueError):
            validar_notas((5, 5, 5))

    def test_validar_notas_requires_three_values(self):
        with self.assertRaises(ValueError):
            validar_notas([5, 5])

    def test_parse_grade_valid(self):
        self.assertEqual(parse_grade("0"), 0.0)
        self.assertEqual(parse_grade("10"), 10.0)
        self.assertAlmostEqual(parse_grade("7.5"), 7.5)

    def test_parse_grade_invalid(self):
        with self.assertRaises(ValueError):
            parse_grade("abc")
        with self.assertRaises(ValueError):
            parse_grade("-1")
        with self.assertRaises(ValueError):
            parse_grade("11")

    def test_determine_result(self):
        self.assertEqual(determine_result(5.0), "Aprobado")
        self.assertEqual(determine_result(4.99), "Suspenso")


if __name__ == "__main__":
    unittest.main()
```

7. Verificación

Prompt

Actúa como Tester.

Revisa `main.py` y `tools_notas.py`.

Comprueba:

1. que `calcular_media` funciona correctamente
2. que `calcular_mediana` funciona correctamente
3. que el programa muestra aprobado si el resultado es >= 5
4. que el programa muestra suspenso si el resultado es < 5

Propón al menos 4 casos de prueba manuales.

Respuesta

Criterios comprobados

- `calcular_media` devuelve el valor esperado.
- `calcular_mediana` devuelve la nota central tras ordenar.
- La clasificación final muestra `Aprobado` o `Suspenso` correctamente.

Casos de prueba manuales

- Ana: `5, 5, 5` -> media `5` -> `Aprobado`
- Luis: `4, 4, 4` -> media `4` -> `Suspenso`
- Marta: `10, 8, 9` -> media `9` -> `Aprobado`
- Pedro: `2, 10, 4` -> mediana `4` -> `Suspenso`

Resultado

El programa cumple los criterios principales y queda respaldado por pruebas automáticas y manuales.

7.1. Ejecución calculando media

```
Introduce el nombre del alumno: Introduce la nota 1: Introduce la nota 2: Introduce la nota 3: ¿Qué quieres calcular
Alumno: Marta
Tipo de cálculo: media
Resultado numérico: 9.0
Resultado: Aprobado
```

7.2. Ejecución calculando mediana

```
Introduce el nombre del alumno: Introduce la nota 1: Introduce la nota 2: Introduce la nota 3: ¿Qué quieres calcular
Alumno: Pedro
Tipo de cálculo: mediana
Resultado numérico: 4.0
Resultado: Suspenso
```

7.3. Resultado de las pruebas unitarias

```
test_calcular_media (test_main.TestToolsNotas.test_calcular_media) ... ok
test_calcular_mediana (test_main.TestToolsNotas.test_calcular_mediana) ... ok
test_determine_result (test_main.TestToolsNotas.test_determine_result) ... ok
test_parse_grade_invalid (test_main.TestToolsNotas.test_parse_grade_invalid) ... ok
test_parse_grade_valid (test_main.TestToolsNotas.test_parse_grade_valid) ... ok
test_validar_notas_requires_list (test_main.TestToolsNotas.test_validar_notas_requires_list) ... ok
test_validar_notas_requires_three_values (test_main.TestToolsNotas.test_validar_notas_requires_three_values) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.000s

OK
```

8. Publisher, GitHub y MCP

Prompt

Actúa como Publisher.

Revisa el proyecto y comprueba que existen:

- `AGENTS.md`
- `PRD.md`
- `SPEC.md`
- `README.md`
- `main.py`
- `tools_notas.py`
- `test_main.py`

Después, usando GitHub MCP sobre el repositorio publicado:

1. comprueba las tools disponibles
2. verifica la identidad de la cuenta en GitHub
3. deja una prueba real sobre el issue `Mejoras futuras`
4. confirma esa acción leyendo el issue

Respuesta

Se revisó la presencia de los archivos principales, se comprobó el repositorio publicado en GitHub y se usó GitHub MCP para verificar la cuenta, añadir un comentario al issue `#1` y comprobar después que ese comentario quedó registrado.

Resultado

El proyecto quedó publicado en GitHub y acompañado por una evidencia real del uso de GitHub MCP sobre el propio repositorio.

8.1. Revisión local del proyecto

```
Revisión del agente Publisher

AGENTS.md: OK
PRD.md: OK
SPEC.md: OK
README.md: OK
main.py: OK
tools_notas.py: OK
test_main.py: OK

Rama local: main

Repositorio público:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp

Issue creado:
```

```
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1

Estado: proyecto publicado y seguimiento creado en GitHub.
```

8.2. Publicación e issue en GitHub

```
Publicación en GitHub ejecutada

Repositorio público:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp

Issue principal:
#1 - Mejoras futuras
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1

Verificación adicional mediante GitHub MCP:
- get_me
- add_issue_comment
- issue_read

Comentario creado por MCP:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1#issuecomment-4346856335
```

8.3. Uso real de GitHub MCP

Servidor utilizado

- Repositorio de referencia: `https://github.com/github/github-mcp-server`
- Endpoint utilizado: `https://api.githubcopilot.com/mcp/`
- Protocolo MCP: `2025-06-18`

Secuencia ejecutada

- `initialize`
- `notifications/initialized`
- `tools/list`
- `get_me`
- `add_issue_comment`
- `issue_read` con `get_comments`

Resultado visible

- Repositorio: `https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp`
- Issue usado: `https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1`
- Comentario creado por MCP: `https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1#issuecomment-4346856335`

Archivos de prueba

- `evidencias/github_mcp_initialize_and_tools.json`
- `evidencias/github_mcp_issue_comment.json`
- `evidencias/github_mcp_issue_comment.txt`

8.4. Registro del comentario creado por MCP

```
Uso real de GitHub MCP verificado

Servidor: github-mcp-server
Endpoint: https://api.githubcopilot.com/mcp/
Repositorio: doriveramirez/calculadora-notas-ia-tools-mcp
Usuario autenticado con get_me: doriveramirez
Issue usado: #1 - Mejoras futuras

Llamadas MCP realizadas:
- initialize -> 200
- notifications/initialized -> 202
- get_me -> 200
- add_issue_comment -> 200
- issue_read (get_comments) -> 200

Comentario creado por MCP:
Verificacion realizada mediante GitHub MCP para la practica de desarrollo con agentes. Fecha: 2026-04-29 20:25:41

URL del comentario:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1#issuecomment-4346856335

Pruebas guardadas:
- evidencias/github_mcp_initialize_and_tools.json
- evidencias/github_mcp_issue_comment.json
```

9. Documentación final

9.1. README

Descripción

Aplicación de consola en Python que solicita el nombre de un alumno y tres notas, permite elegir entre calcular la media o la mediana y muestra si el resultado final es aprobado o suspenso. El proyecto incluye también la publicación del repositorio y una evidencia real del uso de GitHub MCP sobre ese repositorio.

Archivos principales

- `PRD.md`: objetivo y funcionalidades del proyecto.
- `SPEC.md`: comportamiento esperado y criterios de aceptación.
- `AGENTS.md`: definición de los roles usados en la práctica.
- `main.py`: programa principal.
- `tools_notas.py`: funciones externas para calcular media y mediana.
- `test_main.py`: pruebas unitarias.
- `EVIDENCIAS.md`: índice de evidencias.
- `REFLEXION.md`: reflexión final.
- `evidencias/github_mcp_uso.md`: resumen del uso real de GitHub MCP.
- `evidencias/github_mcp_issue_comment.json`: registro de la llamada MCP que añade y verifica un comentario.

Requisitos

- Python 3 instalado

Ejecución

```
python main.py
```

Ejemplo de uso

- Nombre: Marta
- Notas: 10, 8, 9
- Tipo de cálculo: media
- Resultado numérico: 9.0
- Resultado: Aprobado

Trabajo por agentes

- Architect: propuso la estructura del proyecto y la separación de responsabilidades.
- Builder: adaptó `main.py` para usar tools externas.
- Tester: comprobó media, mediana y clasificación final.

- Documenter: organizó la documentación y las evidencias.
- Tool Specialist: creó `tools_notas.py`.
- Publisher: revisó el proyecto, verificó la publicación en GitHub y dejó una prueba real de GitHub MCP sobre el issue `#1`.

Evidencias

Las capturas, registros y pruebas del uso de GitHub MCP están dentro de la carpeta `evidencias/`.

9.2. Reflexión final

Qué se aprende con esta segunda parte

Esta práctica permite ver que un agente no solo redacta texto, sino que puede apoyarse en funciones externas para delegar una parte concreta del trabajo. En este caso, la media y la mediana se sacan fuera del programa principal para que la lógica quede mejor organizada.

Papel de las tools

Separar `calcular_media` y `calcular_mediana` en `tools_notas.py` hace más fácil comprobar el funcionamiento, reutilizar el cálculo y mantener `main.py` más claro.

Papel de la parte de GitHub y MCP

La parte de Publisher sirve para entender que un agente también puede encargarse de la publicación del proyecto y del seguimiento de mejoras en GitHub, dejando constancia del trabajo realizado fuera del código principal. En esta entrega no se quedó solo en la preparación: se usó el servidor GitHub MCP para identificar la cuenta con `get_me`, añadir un comentario al issue `#1` con `add_issue_comment` y comprobarlo después con `issue_read`.

Qué mejoraría

Como mejora futura, añadiría una forma de guardar resultados, más validaciones y más pruebas automáticas, pero manteniendo el proyecto sencillo.

10. Anexo de capturas

En las páginas siguientes se incluyen las capturas completas de los documentos y evidencias de la práctica.

01_prd.png

PRD.md

```
# PRD - Calculadora de notas con Tools y GitHub MCP

## Objetivo
Crear una aplicación de consola en Python que permita introducir el nombre de un alumno
y tres notas, elegir si se quiere calcular la media o la mediana, y mostrar si el result
ado es aprobado o suspenso.

## Usuario
Profesorado o alumnado que quiera calcular rápidamente una nota final usando un flujo de
 trabajo organizado con agentes.

## Funcionalidades
- Pedir el nombre del alumno
- Pedir tres notas numéricas
- Permitir elegir entre media o mediana
- Calcular el resultado numérico usando funciones externas en `tools_notas.py`
- Mostrar el resultado numérico final
- Indicar aprobado si el resultado es mayor o igual que 5
- Indicar suspenso si el resultado es menor que 5
- Publicar el proyecto en GitHub
- Dejar una evidencia real del uso de GitHub MCP sobre el repositorio

## Requisitos técnicos
- Implementación en Python
- Ejecución por consola
- Separación de la lógica de cálculo en un archivo de tools
- Código claro y sencillo
- Documentación del uso de agentes y de GitHub MCP
```

02_spec.png

SPEC.md

```
# SPEC - Cálculo de notas con media, mediana y uso de GitHub MCP

## Problema
Se necesita una aplicación sencilla para calcular la nota final de un alumno a partir de
 tres notas. El usuario podrá elegir si quiere calcular la media o la mediana. Además, e
l proyecto debe quedar publicado en GitHub y acompañado por una prueba real del uso de G
itHub MCP sobre el repositorio.

## Solución esperada
El programa solicitará:
- nombre del alumno
- nota 1
- nota 2
- nota 3
- tipo de cálculo: `media` o `mediana`

Después:
- usará `tools_notas.py` para calcular el resultado numérico
- mostrará el nombre del alumno
- mostrará el tipo de cálculo elegido
- mostrará el resultado numérico
- mostrará `Aprobado` o `Suspenso`

## Archivos principales esperados
- `main.py`
- `tools_notas.py`
- `test_main.py`
- `PRD.md`
- `SPEC.md`
- `AGENTS.md`
- `README.md`

## Criterios de aceptación
- `calcular_media(notas)` devuelve la media correcta
- `calcular_mediana(notas)` devuelve la nota central tras ordenar tres notas
- `main.py` importa y usa `calcular_media` y `calcular_mediana`
- El programa permite elegir entre media o mediana
- Si el resultado numérico es mayor o igual que 5, muestra `Aprobado`
- Si el resultado numérico es menor que 5, muestra `Suspenso`
- El proyecto queda documentado y preparado para GitHub
- La entrega incluye evidencia real del uso de GitHub MCP
```

03_agents.png

AGENTS.md

```
# AGENTS.md

## Reglas generales
- Todos los agentes deben basarse en `PRD.md` y `SPEC.md`.
- No inventar funcionalidades no pedidas.
- Mantener el código simple y claro.
- Explicar brevemente los cambios realizados.

## Agent 1: Architect
Rol:
- Leer `PRD.md` y `SPEC.md`.
- Resumir la solución.
- Proponer la estructura mínima del programa y la separación en archivos.

Salida esperada:
- Explicación breve de la estructura del proyecto.

## Agent 2: Builder
Rol:
- Implementar y modificar `main.py`.
- Seguir exactamente la SPEC.
- Integrar el uso de `tools_notas.py`.

Salida esperada:
- Archivo `main.py` funcional.

## Agent 3: Tester
Rol:
- Revisar si `main.py` y `tools_notas.py` cumplen los criterios de aceptación.
- Proponer casos de prueba manuales.
- Detectar errores o mejoras.

Salida esperada:
- Lista de pruebas y verificación.

## Agent 4: Documenter
Rol:
- Redactar `README.md`.
- Explicar qué hace el programa y cómo ejecutarlo.

Salida esperada:
- Documentación breve y clara del proyecto.

## Agent 5: Tool Specialist
Rol:
- Crear funciones externas para calcular la media y la mediana.
- Separar la lógica de cálculo del programa principal.
- Verificar que las funciones reciben una lista de notas.
- No añadir funcionalidades innecesarias.

Salida esperada:
- Archivo `tools_notas.py`.
- Explicación breve de las funciones creadas.

## Agent 6: Publisher
Rol:
- Preparar el proyecto para publicarlo en GitHub.
- Revisar que existen los archivos principales.
- Usar GitHub MCP sobre el repositorio para dejar una evidencia real.
- Crear un resumen del proyecto.
- Crear y mantener un issue con mejoras futuras.

Salida esperada:
- Comprobación final de archivos.
- Proyecto publicado en GitHub.
- Evidencia del uso de GitHub MCP.
- Issue con mejoras.
```

04_tools_notas.png

tools_notas.py

```python
"""Funciones externas para calcular la media o la mediana de tres notas."""


def validar_notas(notas: list[float]) -> None:
    """Comprueba que las notas llegan en una lista de tres valores numéricos."""
    if not isinstance(notas, list):
        raise ValueError("Las notas deben recibirse en una lista.")

    if len(notas) != 3:
        raise ValueError("Se requieren exactamente tres notas.")

    for nota in notas:
        if not isinstance(nota, (int, float)):
            raise ValueError("Cada nota debe ser numérica.")


def calcular_media(notas: list[float]) -> float:
    """Devuelve la media aritmética de tres notas."""
    validar_notas(notas)
    return sum(notas) / len(notas)


def calcular_mediana(notas: list[float]) -> float:
    """Devuelve la nota central tras ordenar tres notas."""
    validar_notas(notas)
    notas_ordenadas = sorted(notas)
    return notas_ordenadas[1]
```

05_tool_specialist.png

tool_specialist_interaccion.md

```
# Interacción con el agente Tool Specialist

## Prompt
Actúa como Tool Specialist.

Basándote en `PRD.md` y `SPEC.md`, crea un archivo `tools_notas.py` con dos funciones:
1. `calcular_media(notas)`
2. `calcular_mediana(notas)`

Requisitos:
- `notas` será una lista de tres números
- `calcular_media` devolverá la media aritmética
- `calcular_mediana` devolverá la nota central después de ordenar
- código sencillo
- sin funcionalidades extra

## Respuesta
Se creó `tools_notas.py` con:
- una función `validar_notas(notas)` para comprobar que llega una lista de tres valores
numéricos
- `calcular_media(notas)` para devolver la media
- `calcular_mediana(notas)` para devolver la nota central

## Resultado
La lógica de cálculo quedó separada del programa principal, tal como pedía el enunciado.
```

06_builder.png

builder_interaccion.md

```
# Interacción con el agente Builder

## Prompt
Actúa como Builder.

Modifica `main.py` para usar las funciones del archivo `tools_notas.py`.

Requisitos:
- importar `calcular_media` y `calcular_mediana`
- pedir nombre del alumno
- pedir tres notas
- preguntar si se quiere calcular media o mediana
- mostrar el resultado numérico
- indicar aprobado si el resultado es mayor o igual que 5
- indicar suspenso si es menor que 5
- mantener el código sencillo

## Respuesta
Se adaptó `main.py` para:
- importar las funciones externas
- pedir tres notas por consola
- preguntar el tipo de cálculo
- delegar el cálculo en `tools_notas.py`
- mostrar el resultado final

## Resultado
`main.py` quedó conectado con `tools_notas.py` y cumple la SPEC.
```

07_programa_media.png

ejecucion_media.txt

```
Introduce el nombre del alumno: Introduce la nota 1: Introduce la nota 2: Introduce la n
ota 3: ¿Qué quieres calcular? media/mediana:
Alumno: Marta
Tipo de cálculo: media
Resultado numérico: 9.0
Resultado: Aprobado
```

08_programa_mediana.png

ejecucion_mediana.txt

```
Introduce el nombre del alumno: Introduce la nota 1: Introduce la nota 2: Introduce la n
ota 3: ¿Qué quieres calcular? media/mediana:
Alumno: Pedro
Tipo de cálculo: mediana
Resultado numérico: 4.0
Resultado: Suspenso
```

09_tester.png

tester_interaccion.md

```md
# Interacción con el agente Tester

## Prompt
Actúa como Tester.

Revisa `main.py` y `tools_notas.py`.

Comprueba:
1. que `calcular_media` funciona correctamente
2. que `calcular_mediana` funciona correctamente
3. que el programa muestra aprobado si el resultado es >= 5
4. que el programa muestra suspenso si el resultado es < 5

Propón al menos 4 casos de prueba manuales.

## Respuesta
### Criterios comprobados
- `calcular_media` devuelve el valor esperado.
- `calcular_mediana` devuelve la nota central tras ordenar.
- La clasificación final muestra `Aprobado` o `Suspenso` correctamente.

### Casos de prueba manuales
- Ana: `5, 5, 5` -> media `5` -> `Aprobado`
- Luis: `4, 4, 4` -> media `4` -> `Suspenso`
- Marta: `10, 8, 9` -> media `9` -> `Aprobado`
- Pedro: `2, 10, 4` -> mediana `4` -> `Suspenso`

## Resultado
El programa cumple los criterios principales y queda respaldado por pruebas automáticas
y manuales.
```

10_publisher.png

publisher_interaccion.md

```
# Interacción con el agente Publisher

## Prompt
Actúa como Publisher.

Revisa el proyecto y comprueba que existen:
- `AGENTS.md`
- `PRD.md`
- `SPEC.md`
- `README.md`
- `main.py`
- `tools_notas.py`
- `test_main.py`

Después, usando GitHub MCP sobre el repositorio publicado:
1. comprueba las tools disponibles
2. verifica la identidad de la cuenta en GitHub
3. deja una prueba real sobre el issue `Mejoras futuras`
4. confirma esa acción leyendo el issue

## Respuesta
Se revisó la presencia de los archivos principales, se comprobó el repositorio publicado
 en GitHub y se usó GitHub MCP para verificar la cuenta, añadir un comentario al issue `
#1` y comprobar después que ese comentario quedó registrado.

## Resultado
El proyecto quedó publicado en GitHub y acompañado por una evidencia real del uso de Git
Hub MCP sobre el propio repositorio.
```

11_github_mcp_uso.png

```
github_mcp_uso.md

# Uso real de GitHub MCP

## Servidor utilizado
- Repositorio de referencia: `https://github.com/github/github-mcp-server`
- Endpoint utilizado: `https://api.githubcopilot.com/mcp/`
- Protocolo MCP: `2025-06-18`

## Secuencia ejecutada
- `initialize`
- `notifications/initialized`
- `tools/list`
- `get_me`
- `add_issue_comment`
- `issue_read` con `get_comments`

## Resultado visible
- Repositorio: `https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp`
- Issue usado: `https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1
`
- Comentario creado por MCP: `https://github.com/doriveramirez/calculadora-notas-ia-tool
s-mcp/issues/1#issuecomment-4346856335`

## Archivos de prueba
- `evidencias/github_mcp_initialize_and_tools.json`
- `evidencias/github_mcp_issue_comment.json`
- `evidencias/github_mcp_issue_comment.txt`
```

12_readme.png

README.md

```
# Calculadora de notas con Tools y GitHub MCP

## Descripción
Aplicación de consola en Python que solicita el nombre de un alumno y tres notas, permit
e elegir entre calcular la media o la mediana y muestra si el resultado final es aprobad
o o suspenso. El proyecto incluye también la publicación del repositorio y una evidencia
 real del uso de GitHub MCP sobre ese repositorio.

## Archivos principales
- `PRD.md`: objetivo y funcionalidades del proyecto.
- `SPEC.md`: comportamiento esperado y criterios de aceptación.
- `AGENTS.md`: definición de los roles usados en la práctica.
- `main.py`: programa principal.
- `tools_notas.py`: funciones externas para calcular media y mediana.
- `test_main.py`: pruebas unitarias.
- `EVIDENCIAS.md`: índice de evidencias.
- `REFLEXION.md`: reflexión final.
- `evidencias/github_mcp_uso.md`: resumen del uso real de GitHub MCP.
- `evidencias/github_mcp_issue_comment.json`: registro de la llamada MCP que añade y ver
ifica un comentario.

## Requisitos
- Python 3 instalado

## Ejecución
```bash
python main.py
```

Ejemplo de uso
- Nombre: Marta
- Notas: 10, 8, 9
- Tipo de cálculo: media
- Resultado numérico: 9.0
- Resultado: Aprobado

Trabajo por agentes
- Architect: propuso la estructura del proyecto y la separación de responsabilidades.
- Builder: adaptó `main.py` para usar tools externas.
- Tester: comprobó media, mediana y clasificación final.
- Documenter: organizó la documentación y las evidencias.
- Tool Specialist: creó `tools_notas.py`.
- Publisher: revisó el proyecto, verificó la publicación en GitHub y dejó una prueba rea
l de GitHub MCP sobre el issue `#1`.

Evidencias
Las capturas, registros y pruebas del uso de GitHub MCP están dentro de la carpeta `evid
encias/`.
```

13_revision_archivos_publisher.png

revision_archivos_publisher.txt

```
Revisión del agente Publisher

AGENTS.md: OK
PRD.md: OK
SPEC.md: OK
README.md: OK
main.py: OK
tools_notas.py: OK
test_main.py: OK

Rama local: main

Repositorio público:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp

Issue creado:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1

Estado: proyecto publicado y seguimiento creado en GitHub.
```

14_pruebas_unitarias.png

pruebas_unitarias.txt

```
test_calcular_media (test_main.TestToolsNotas.test_calcular_media) ... ok
test_calcular_mediana (test_main.TestToolsNotas.test_calcular_mediana) ... ok
test_determine_result (test_main.TestToolsNotas.test_determine_result) ... ok
test_parse_grade_invalid (test_main.TestToolsNotas.test_parse_grade_invalid) ... ok
test_parse_grade_valid (test_main.TestToolsNotas.test_parse_grade_valid) ... ok
test_validar_notas_requires_list (test_main.TestToolsNotas.test_validar_notas_requires_l
ist) ... ok
test_validar_notas_requires_three_values (test_main.TestToolsNotas.test_validar_notas_re
quires_three_values) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.000s

OK
```

15_github_publicacion_issue.png

```
github_publicacion_y_issue.txt

Publicación en GitHub ejecutada

Repositorio público:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp

Issue principal:
#1 - Mejoras futuras
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1

Verificación adicional mediante GitHub MCP:
- get_me
- add_issue_comment
- issue_read

Comentario creado por MCP:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1#issuecomment-43
46856335
```

16_github_mcp_issue_comment.png

```
github_mcp_issue_comment.txt

Uso real de GitHub MCP verificado

Servidor: github-mcp-server
Endpoint: https://api.githubcopilot.com/mcp/
Repositorio: doriveramirez/calculadora-notas-ia-tools-mcp
Usuario autenticado con get_me: doriveramirez
Issue usado: #1 - Mejoras futuras

Llamadas MCP realizadas:
- initialize -> 200
- notifications/initialized -> 202
- get_me -> 200
- add_issue_comment -> 200
- issue_read (get_comments) -> 200

Comentario creado por MCP:
Verificacion realizada mediante GitHub MCP para la practica de desarrollo con agentes. F
echa: 2026-04-29 20:25:41

URL del comentario:
https://github.com/doriveramirez/calculadora-notas-ia-tools-mcp/issues/1#issuecomment-43
46856335

Pruebas guardadas:
- evidencias/github_mcp_initialize_and_tools.json
- evidencias/github_mcp_issue_comment.json
```